

Automated SSH Brute Force Detection and Response using Splunk SIEM & Shuffle SOAR

Phase 1: Splunk SIEM Setup

Step 1: Created Splunk Server VM

What We Did

Created a dedicated Ubuntu Server virtual machine to host Splunk Enterprise.

Configuration

Parameter	Value
VM Name	splunk-server
IP Address	192.168.0.5
OS	Ubuntu Server
Purpose	SIEM Server

Commands Used

No command required. VM was created using the virtualization platform.

Why We Did This

A dedicated SIEM server is required to collect, store, analyze, and monitor security logs from different systems across the network.

Step 2: Installed Splunk Enterprise

What We Did

Downloaded and installed Splunk Enterprise on the Ubuntu server.

Command Used

```
sudo dpkg -i splunk*.deb
```

Explanation of Command

- dpkg → Debian package installer

- `-i` → Install package
- `splunk*.deb` → Splunk installation package

Why We Used This Command

This command installs Splunk Enterprise so that the server can ingest logs, perform searches, generate alerts, and act as a SIEM platform.

Step 3: Started Splunk Service

What We Did

Started the Splunk service and verified its status.

Commands Used

```
sudo /opt/splunk/bin/splunk start --run-as-root
```

Verification:

```
sudo /opt/splunk/bin/splunk status
```

Explanation of Commands

Start Command

```
splunk start --run-as-root
```

- Starts Splunk daemon (`splunkd`)
- `--run-as-root` allows Splunk to start under root privileges

Status Command

```
splunk status
```

Checks whether the Splunk service is running correctly.

Why We Used These Commands

Splunk must be running before it can receive logs, create alerts, or provide access to the web interface.

Step 4: Accessed Splunk Web UI

What We Did

Accessed the Splunk management interface through a web browser.

URL

```
http://49.50.92.78:8000
```

Credentials

Username: admin

Password: splunk@123

Why We Did This

The Splunk Web Interface allows administrators to:

- Search logs
 - Create alerts
 - Build dashboards
 - Configure forwarding and receiving
-

Step 5: Enabled Receiving Port

What We Did

Configured Splunk to receive logs from Universal Forwarders.

Configuration Path

Settings

→ Forwarding and Receiving

→ Configure Receiving

→ New Receiving Port

→ 9997

Verification Command

```
sudo netstat -tulnp | grep 9997
```

Explanation of Command

```
netstat -tulnp
```

Shows all listening network ports.

```
grep 9997
```

Filters output to show only port 9997.

Why We Used This Command

To verify that Splunk is actively listening for incoming logs from forwarders.

Expected Output:

```
0.0.0.0:9997 LISTEN
```

Phase 2: Target Machine Setup

Step 6: Created Ubuntu Target VM

What We Did

Created a second Ubuntu virtual machine that would act as the victim server.

Configuration

Parameter	Value
VM Name	target-server
IP Address	192.168.0.6
Purpose	Victim Machine

Why We Did This

The target machine generates authentication logs that will later be analyzed by Splunk.

Step 7: Installed OpenSSH Server

What We Did

Installed SSH service on the target machine.

Commands Used

```
sudo apt update  
sudo apt install openssh-server -y
```

Verification:

```
sudo systemctl status ssh
```

Explanation of Commands

apt update

Updates package repository information.

apt install openssh-server

Installs SSH server software.

systemctl status ssh

Checks whether SSH service is running.

Why We Used These Commands

The SSH server is the service targeted during brute-force attack simulation and generates authentication logs required for detection.

Step 8: Created Test User

What We Did

Created a user account for brute-force attack simulation.

Command Used

```
sudo adduser testuser
```

Explanation of Command

Creates a new Linux user named:

```
testuser
```

Why We Used This Command

Hydra requires a target account to attempt authentication against. This user was created specifically for testing purposes.

Step 9: Verified SSH Logging

What We Did

Verified that SSH authentication events are being written to system logs.

Command Used

```
sudo tail -f /var/log/auth.log
```

Explanation of Command

```
tail -f
```

Continuously displays newly added log entries.

Why We Used This Command

To ensure failed and successful login attempts generate logs that can later be forwarded to Splunk.

Observed Events:

```
Failed password
```

```
Accepted password
```

Phase 3: Log Forwarding Setup

Step 10: Installed Splunk Universal Forwarder

What We Did

Installed Splunk Universal Forwarder on the target server.

Purpose

Forward security logs to Splunk Enterprise.

Why We Did This

The Universal Forwarder acts as an agent that continuously sends logs from Linux systems to the Splunk SIEM server.

Step 11: Connected Forwarder to Splunk

What We Did

Configured the Universal Forwarder to send logs to Splunk.

Command Used

```
sudo /opt/splunkforwarder/bin/splunk add forward-server 192.168.0.5:9997
```

Explanation of Command

add forward-server

Registers the Splunk server as a log receiver.

192.168.0.5:9997

Splunk Enterprise IP and receiving port.

Why We Used This Command

To establish communication between the target machine and the SIEM server.

Step 12: Added Log Monitoring

What We Did

Configured the forwarder to monitor SSH authentication logs.

Command Used

```
sudo /opt/splunkforwarder/bin/splunk add monitor /var/log/auth.log
```

Explanation of Command

add monitor

Tells Splunk Forwarder to watch a specific file.

/var/log/auth.log

Linux authentication log file.

Why We Used This Command

This ensures all SSH authentication events are automatically forwarded to Splunk.

Step 13: Verified Logs in Splunk

What We Did

Verified that logs from the target machine were reaching Splunk.

Search Query

index=*

Explanation of Query

Searches all available indexes for events.

Why We Used This Query

To confirm successful log ingestion.

Verified Fields:

host=target-server
source=/var/log/auth.log

Phase 4: Attacker Machine Setup

Step 14: Created Attacker VM

What We Did

Created a dedicated machine for attack simulation.

Configuration

Parameter	Value
VM Name	attacker-server
IP Address	192.168.0.7
Purpose	Attack Simulation

Why We Did This

Separating attacker and victim systems creates a realistic SOC environment.

Step 15: Installed Hydra

What We Did

Installed Hydra on the attacker machine.

Command Used

```
sudo apt install hydra -y
```

Explanation of Command

Installs Hydra password auditing tool.

Why We Used This Command

Hydra was used to generate SSH authentication failures that would later be detected by Splunk.

Step 16: Investigated SSH Port Configuration

What We Did

Troubleshoot SSH connectivity issues.

Issue Encountered

Connection Refused

Investigation Command

```
sudo ss -tulnp | grep ssh
```

Explanation of Command

```
ss -tulnp
```

Displays active listening ports and services.

grep ssh

Filters output to SSH service only.

Result

Found SSH listening on:

Port 2232

instead of:

Port 22

Why We Used This Command

To identify the actual SSH listening port and troubleshoot connection failures during attack simulation.

Phase 5: SSH Brute Force Detection in Splunk

Step 17: Generated SSH Brute Force Logs

What We Did

Generated multiple failed SSH authentication attempts against the target server to simulate a brute-force attack.

Activity Performed

Repeated authentication attempts were made from the attacker machine against the target server.

Verification Command

```
sudo tail -f /var/log/auth.log
```

Explanation of Command

```
tail -f
```

Continuously monitors log entries as they are generated.

/var/log/auth.log

Linux authentication log file containing SSH login activity.

Why We Used This Command

To verify that failed SSH login attempts were successfully generating security events that could later be analyzed by Splunk.

Observed Events

```
Failed password for testuser  
Failed password for invalid user
```

Step 18: Verified Attack Logs in Splunk

What We Did

Confirmed that SSH authentication failure logs were successfully forwarded to Splunk.

Search Query

```
index=* "Failed password"
```

Explanation of Query

Searches all indexed events containing the phrase:

```
Failed password
```

Why We Used This Query

To verify that authentication failure logs generated on the target machine were reaching the SIEM platform.

Verification

```
host=target-server  
source=/var/log/auth.log  
sourcetype=syslog
```

Step 19: Created SSH Brute Force Detection Query

What We Did

Created a detection query to identify failed SSH authentication attempts.

Detection Query

```
index=* "Failed password"
```

Explanation of Query

Searches all events containing failed password attempts.

Why We Used This Query

Failed SSH login attempts are one of the primary indicators of brute-force attacks. This query forms the basis of our detection logic.

Step 20: Created Real-Time Alert

What We Did

Configured a Splunk alert to automatically trigger whenever the detection query finds matching events.

Alert Configuration

Alert Name : SSH Brute Force Detection
Alert Type : Real-Time
Trigger : Per Result

Explanation

Real-Time Alert

Continuously monitors incoming events.

Per Result Trigger

Triggers the alert every time a matching event is detected.

Why We Used This Configuration

To ensure immediate detection and response whenever suspicious SSH authentication activity occurs.

Phase 6: Shuffle SOAR Integration

Step 21: Deployed Shuffle SOAR

What We Did

Created a dedicated virtual machine to host the Shuffle SOAR platform.

Configuration

Parameter	Value
-----------	-------

VM Name	Server-SOAR
---------	-------------

Parameter	Value
IP Address	192.168.0.8
Purpose	Security Orchestration and Automation

Why We Did This

Shuffle SOAR is responsible for automating incident response actions after detection by Splunk.

Step 22: Installed Shuffle Using Docker

What We Did

Installed and deployed Shuffle SOAR using Docker containers.

Verification Command

```
docker ps
```

Explanation of Command

```
docker ps
```

Displays all currently running Docker containers.

Why We Used This Command

To verify that all required Shuffle services were running correctly.

Verified Containers

```
shuffle-backend  
shuffle-frontend  
shuffle-worker  
shuffle-orborus  
shuffle-opensearch
```

Step 23: Accessed Shuffle Web Interface

What We Did

Accessed the Shuffle dashboard through a web browser.

URL

```
http://49.50.92.86:3001
```

Activity Performed

Created administrator account and logged into Shuffle.

Why We Did This

The web interface is required for:

- Creating workflows
- Managing webhooks
- Monitoring executions
- Configuring response actions

Step 24: Created SSH-Bruteforce-Response Workflow

What We Did

Designed an automated workflow inside Shuffle.

Workflow Structure

Webhook Trigger



Python Action



Email Notification

Explanation

Webhook Trigger

Receives alerts from Splunk.

Python Action

Processes incoming event data.

Email Notification

Sends alert information to security analysts.

Why We Created This Workflow

To automate the response process after detection.

Step 25: Generated Webhook Endpoint

What We Did

Created a webhook endpoint inside Shuffle.

Example Webhook

`http://49.50.92.86:3001/api/v1/hooks/webhook_xxxxxxxx`

Why We Generated This Webhook

The webhook serves as the communication channel between Splunk and Shuffle.

Whenever Splunk detects an attack, it sends event data to this endpoint.

Step 26: Integrated Splunk Alert with Shuffle

What We Did

Configured Splunk to send alerts directly to Shuffle.

Configuration

Splunk Alert Action:

Webhook

Webhook URL:

Shuffle Webhook Endpoint

Why We Did This

To establish automatic communication between the SIEM and SOAR platforms.

Result

Whenever Splunk generated an alert, Shuffle automatically received and processed the event.

Step 27: Tested Webhook Connectivity

What We Did

Manually tested webhook communication.

Command Used

```
curl -X POST http://49.50.92.86:3001/api/v1/hooks/webhook_XXXXXX
```

Explanation of Command

curl

Used to send HTTP requests.

-X POST

Sends data using HTTP POST method.

Why We Used This Command

To verify that Shuffle could successfully receive webhook requests before integrating with Splunk.

Result

```
{  
  "success": true  
}
```

Workflow status:

FINISHED

Phase 7: Automated Email Response

Step 28: Added Email Automation Node

What We Did

Added an email action inside the Shuffle workflow.

Purpose

Send automated security notifications whenever an SSH brute-force attack is detected.

Why We Added This Node

Security analysts need immediate visibility of incidents. Email notifications provide a simple and effective alerting mechanism.

Step 29: Configured Gmail SMTP

What We Did

Configured Gmail SMTP as the email delivery service.

Configuration

```
SMTP Host : smtp.gmail.com  
SMTP Port : 587  
TLS/SSL   : Enabled
```

Why We Used Gmail SMTP

Gmail SMTP provides reliable email delivery and allows Shuffle to send automated notifications externally.

Connectivity Verification

```
nc -vz smtp.gmail.com 587
```

Explanation of Command

```
nc
```

Network connectivity testing tool.

```
-vz
```

Tests whether a port is reachable.

Why We Used This Command

To verify that the SOAR server could communicate with Gmail's SMTP service.

Step 30: Generated Gmail App Password

What We Did

Created an App Password for Gmail authentication.

Issue Encountered

```
530 Authentication Required
```

Root Cause

Google blocks authentication using normal account passwords for third-party applications.

Resolution

1. Enabled Two-Factor Authentication (2FA)
2. Generated Gmail App Password
3. Updated Shuffle Email Configuration

Why We Did This

Shuffle requires secure authentication to send emails through Gmail SMTP.

Using App Passwords is Google's recommended method.

Step 31: Tested Email Notification

What We Did

Performed end-to-end validation of the complete workflow.

Test Method

Generated an alert and allowed Shuffle to execute the email action.

Verification

Workflow Execution:

FINISHED

Email Received:

 SSH Brute Force Attack Detected

Why We Performed This Test

To verify the complete workflow:

```
Attack
  ↓
Log Generation
  ↓
Splunk Detection
  ↓
Real-Time Alert
  ↓
Webhook
  ↓
Shuffle SOAR
  ↓
Email Notification
```

Outcome

Successfully validated the entire SIEM-to-SOAR pipeline and confirmed that security notifications were delivered automatically to the analyst.